

Codex Token Efficiency Skill Research Baseline

What the current research supports

The strongest baseline for a Codex “token efficiency” skill is not “always use a cheaper model.” It is “use the most expensive reasoning only where it pays for itself, and force the rest of the work into cheaper, narrower, graph-informed sub-tasks.” OpenAI’s current Codex docs explicitly support this pattern: Codex can spawn subagents in parallel, collect their results, and let you define custom agents with different model configurations and instructions depending on the task. OpenAI’s own guidance also says distinct, separable work tends to perform best when broken into separate turns or workers rather than handled as one monolithic reasoning pass. ¹

For model naming, there is an important normalization step. Public OpenAI docs currently expose `gpt-5.5`, `gpt-5.4`, `gpt-5.4-mini`, `gpt-5.4-nano`, `gpt-5.3-codex-spark`, and some older API models such as `gpt-5.3-codex`, `gpt-5`, `gpt-5-mini`, and `gpt-5-nano`. I did **not** find an official public model slug called “5.4 light.” The closest public equivalents are `gpt-5.4-mini` for general lightweight subagent work and `gpt-5.4-nano` for highly mechanical classification, extraction, ranking, and atomic worker tasks. Likewise, “5.5 Extra High” maps cleanly to `gpt-5.5` with `reasoning.effort: xhigh`, which OpenAI recommends for the hardest asynchronous agentic tasks or eval-like workloads. ²

The other big conclusion is that your graph workflow should be a **first-class efficiency primitive**, not a convenience add-on. The active Graphify repository says it maps a project into a queryable knowledge graph, generates `graph.html`, `GRAPH_REPORT.md`, and `graph.json`, and for Codex specifically it installs guidance into `AGENTS.md` plus a `.codex/hooks.json` pre-tool hook that nudges the agent toward graph queries before raw search or file-reading patterns. Graphify also supports `graphify query`, `graphify path`, `graphify explain`, `graphify hook install`, and update modes like `/graphify ./raw --update` and `graphify update ./src`. That makes it well suited to the exact practice you described: use the graph skill to explore the repo efficiently, then refresh it after changes so the next run does not re-pay the discovery cost. ³

The model routing stack you should hard-code

OpenAI’s current Codex guidance is fairly clear about the top-level routing. For most demanding Codex tasks, start with `gpt-5.5`; for lighter, lower-cost subagent work, use `gpt-5.4-mini`; and for near-instant text-only iteration, `gpt-5.3-codex-spark` is available in research preview for ChatGPT Pro users. Codex also allows you to steer or pin the model and reasoning effort directly in agent definitions, and its subagent docs say the platform may itself favor `gpt-5.4-mini` for fast scans and higher-effort `gpt-5.5` for more demanding reasoning if you do not pin settings manually. ⁴

That leads to a practical alias map for your skill:

Alias to use in your skill	Public model / setting	Best default use	Why this mapping is defensible
<code>frontier_deep</code>	<code>gpt-5.5</code> + <code>xhigh</code>	ambiguous architecture, hard debugging, security review, cross-cutting refactors, coordinator of last resort	<code>gpt-5.5</code> is the strongest current Codex model for demanding work, and <code>xhigh</code> is the documented setting for the hardest asynchronous agentic tasks. ⁵
<code>frontier_balanced</code>	<code>gpt-5.5</code> + <code>medium</code> or <code>high</code>	master coordinator, synthesis, merge of subagent outputs, final plan and final answer	<code>medium</code> is the documented default and balanced starting point; <code>high</code> is better when assumptions and edge cases matter. ⁶
<code>standard_balanced</code>	<code>gpt-5.4</code> + <code>medium</code>	non-trivial coding workflows pinned away from 5.5 for budget reasons	Codex subagent docs position <code>gpt-5.4</code> as the lower-cost strong model for coding, reasoning, tool use, and broader workflows. ⁷
<code>light_worker</code>	<code>gpt-5.4-mini</code> + <code>low</code>	scans, repo exploration, read-heavy review, large-file skim, supporting-document processing, routine subagents	OpenAI explicitly describes <code>gpt-5.4-mini</code> as fast, efficient, and strong for subagents and high-volume workloads. ⁸
<code>atomic_worker</code>	<code>gpt-5.4-nano</code> + <code>none</code> or <code>low</code>	classification, extraction, ranking, de-duplication, chunk triage, rename maps, test failure categorization	OpenAI describes <code>gpt-5.4-nano</code> as optimized for speed/cost-sensitive classification, extraction, ranking, and sub-agents. ⁹
<code>realtime_editor</code>	<code>gpt-5.3-codex-spark</code>	interactive UI tweaks, short edit loops, low-latency code iteration, “do one focused thing now”	OpenAI describes Spark as a smaller real-time coding model for near-instant iteration, text-only, with a lightweight working style and minimal targeted edits. ¹⁰

One extra nuance matters if you use ChatGPT sign-in versus API-key auth. OpenAI’s current Codex model page says `gpt-5.3-codex` and `gpt-5.2` are deprecated in Codex when you sign in with ChatGPT, but that some deprecated ChatGPT-sign-in models can still exist in the API. So if your skill is intended for **Codex with ChatGPT auth**, do not build new routing policies around `gpt-5.3-codex`; if your skill is intended for **Codex with API-key auth**, it can still be a valid lower-cost agentic coding option. ¹¹

The economics behind the coordinator pattern

If your goal is to let a “5.5 extra high” coordinator fan work out to smaller workers, the public cost picture strongly supports that architecture. On the current public model pages, `gpt-5.5` batch/API pricing is \$5 input, \$0.50 cached input, and \$30 output per 1M tokens; `gpt-5.4` is \$2.50 / \$0.25 / \$15; `gpt-5.4-mini` is \$0.75 / \$0.075 / \$4.50; and `gpt-5.4-nano` is \$0.20 / \$0.02 / \$1.25. In Codex credit pricing, `gpt-5.5` is 125 / 12.5 / 750 credits per 1M input / cached input / output tokens, `gpt-5.4` is 62.5 / 6.25 / 375, and `gpt-5.4-mini` is 18.75 / 1.875 / 113. Spark is in research preview with separate limits and was not available in the API at launch. ¹²

The ratios are the key design input for your skill. `gpt-5.4` is about **2x cheaper** than `gpt-5.5` on both input and output; `gpt-5.4-mini` is about **6.67x cheaper** on both input and output; and `gpt-5.4-nano` is about **25x cheaper on input** and **24x cheaper on output**. In credit terms, the same broad relationship holds: `gpt-5.5` versus `gpt-5.4` is roughly 2x, and `gpt-5.5` versus `gpt-5.4-mini` is roughly 6.6x on both major token buckets. That means your coordinator can often afford several mini workers, or a large number of nano workers, before it reaches the spend of one comparable 5.5-heavy run. ¹³

The simplest hard-coded budgeting rule is therefore:

```
expected_cost = input_tokens * input_rate + cached_input_tokens * cached_rate +  
output_tokens * output_rate
```

Then route to the cheapest plan that clears your quality bar. As a starting heuristic, let the 5.5 coordinator ask: “Can I replace one broad 5.5 exploration pass with several graph-scoped mini or nano workers whose *combined* estimated spend stays below 70% to 85% of the predicted 5.5 spend?” That 70%–85% guardband is my recommendation rather than an OpenAI rule, but it is grounded in the large public rate gap between 5.5 and the smaller models. ¹⁴

For API-key workflows, there is one more high-leverage lever: prompt caching. OpenAI’s docs state that prompt caching works automatically, can reduce latency by up to 80% and input-token cost by up to 90% for eligible repeated prefixes, and gets better cache hit rates when stable content is kept at the beginning of the request while dynamic user-specific context is pushed toward the end. If your skill bakes in a stable instruction prefix and a stable graph-query contract, the master coordinator and repeat worker types can benefit from those cache rules. ¹⁵

Prompt design rules by model class

Your instinct that smaller models need more direct language is supported by OpenAI’s prompt guidance. The docs say `gpt-5.4-mini` and `gpt-5.4-nano` are highly steerable but less likely than larger models to infer missing steps or resolve ambiguity implicitly, and recommend putting critical rules first, specifying full execution order when side effects matter, using structural scaffolding such as numbered steps and explicit decision rules, and separating “do the action” from “report the action.” ¹⁶

For `gpt-5.5`, the best current guidance is almost the opposite. OpenAI now recommends shorter, outcome-first prompts rather than process-heavy legacy prompt stacks. Their docs explicitly say GPT-5.5 works best when prompts define the outcome and leave room for the model to choose an efficient path,

and warn that carrying forward overly heavy legacy instructions can add noise, narrow search space, and make outputs overly mechanical. They also recommend re-evaluating `low` and `medium` reasoning before escalating, since GPT-5.5 is more token-efficient than prior generations at similar reasoning settings. ¹⁷

The practical inference for your skill is this: use **short, goal-led prompts for 5.5**, but use **narrow, structured task cards for mini and nano**. Smaller models may need more explicit prompts, but that does **not** mean giving them long, open-ended prompts. Instead, it means giving them a smaller scope, a stricter output contract, and fewer degrees of freedom. That inference follows directly from OpenAI's small-model guidance plus Codex's recommendation that mini workers are best for fast scans, read-heavy review, and distilled parallel work returned to a main agent. ¹⁸

For reasoning effort, Codex's subagent docs currently recommend `medium` as a balanced default, `low` when the task is straightforward and speed matters most, and `high` when an agent must trace complex logic, check assumptions, or work through edge cases such as reviewer or security-focused tasks. GPT-5.5 additionally supports `xhigh`, which the latest OpenAI guide reserves for the hardest asynchronous agentic tasks. That gives you a good hard-coded ladder: `none` or `low` for atomic workers, `low` for scan workers, `medium` for most coordinators, `high` for reviewers and cross-file reasoning, and `xhigh` only for the truly hard cases. ¹⁹

The graph-first workflow should be mandatory

Graphify is especially relevant because its whole purpose is to reduce the need to re-read raw repositories. The current repository README says `/graphify .` maps code, docs, PDFs, images, and videos into a knowledge graph; the output includes a browsable HTML graph, a full `graph.json`, and a `GRAPH_REPORT.md`; it provides commands such as `graphify query`, `graphify path`, and `graphify explain`; and for Codex it writes query-first guidance into `AGENTS.md` and installs a `.codex/hooks.json` pre-tool hook. The repo also documents `graphify hook install` for post-commit auto-rebuilds and update modes such as `/graphify ./raw --update` and `graphify update ./src`. ³

That means the first practice in your skill should be phrased as a hard rule, not a polite suggestion: **graph before grep, graph before broad file reads, graph before spawning multiple workers**. A coordinator should first use the graph to produce a minimal work plan: relevant communities, connected symbols, likely blast radius, and the smallest file set worth opening. Then it should only hand each worker the graph slice and exact file list needed for that worker. This is partly my recommendation, but it is directly aligned with Graphify's own query-first guidance and Codex's support for graph-biased hooks. ²⁰

The second mandatory rule is freshness. Graphify documents both commit-hook auto-rebuilds and changed-file update flows, including a recommendation to run `graphify hook install` so the graph rebuilds after commits and `/graphify --update` when docs or papers change. For your use case, the correct policy is: after every implemented change, update the graph or confirm the hook did it for you, then treat the graph as the map for subsequent runs. Otherwise the graph becomes stale and its token savings decay over time. ²¹

Audit checklist for a repo or Codex workflow

Use this as the baseline checklist when auditing whether a repository or Codex setup is likely wasting tokens. The logic is grounded in current Codex subagent/model guidance, OpenAI prompt guidance, prompt caching guidance, and Graphify's query-first/update workflow. ²²

Routing and model selection

- [] Is `gpt-5.5` reserved for coordination, ambiguity resolution, synthesis, validation, or hard cross-cutting work rather than routine repo exploration?
- [] Is there an explicit alias map that treats "5.5 extra high" as `gpt-5.5` + `xhigh`, and "5.4 light" as `gpt-5.4-mini` by default, falling to `gpt-5.4-nano` for purely mechanical tasks?
- [] Are exploration, inventory, file triage, rename maps, test-failure bucketing, and basic extraction delegated to mini or nano workers?
- [] Is Spark used only for fast, focused, text-only iteration where latency matters more than broad reasoning?
- [] If ChatGPT sign-in is used, is the workflow avoiding deprecated ChatGPT-sign-in Codex models such as `gpt-5.3-codex`?
- [] Does the coordinator estimate cost before spawning 3–5 workers instead of assuming parallelism is automatically cheaper?

Prompt discipline

- [] Do `gpt-5.5` prompts lead with the desired outcome, constraints, evidence rules, and final output shape, without carrying forward unnecessary legacy process scaffolding?
- [] Do mini and nano workers receive narrow, structured task cards with critical rules first, explicit execution order, and strict output contracts?
- [] Are prompts free of contradictory instructions that force the model to burn reasoning tokens reconciling conflicts?
- [] Are "do the work" and "report the work" separated for smaller workers?
- [] Are workers forbidden from asking unnecessary follow-up questions when the scope is already sufficiently specified?

Graph usage

- [] Does every exploratory run begin with `graphify query`, `graphify path`, or `graphify explain` before `grep` or broad file reads?
- [] Does the coordinator use graph outputs to define the smallest relevant file set for each worker?
- [] Are workers prevented from reading the same broad file set unless the overlap is intentional?
- [] Is `graphify hook install` enabled so the graph refreshes automatically after commits where possible?
- [] After code, docs, or architecture changes, is the graph explicitly refreshed with an update path if the hook did not already do it?

Context and caching

- [] Is stable instruction content placed at the beginning of requests and dynamic user-specific detail pushed toward the end to improve prompt cache hits?

- [] If API-key auth is used, is `prompt_cache_key` used consistently on repeated worker archetypes?
- [] Are workers given compact graph slices and precise file lists instead of entire reports, folders, or long chat histories?
- [] Are repeated repository norms stored in `AGENTS.md` or a skill instead of re-sent ad hoc in every prompt?
- [] Is the combined instruction layer small enough to avoid wasting context budget on boilerplate?

Quality gates

- [] Does the coordinator escalate to `gpt-5.5` only when a cheaper worker reports ambiguity, conflicting evidence, cross-file coupling, or security-sensitive logic?
- [] Are cheap workers returning distilled findings rather than verbose narratives?
- [] Does the coordinator merge and verify subagent outputs before presenting a final answer or implementing a multi-file change?
- [] After edits, does the workflow run the smallest relevant validation rather than a blanket expensive pipeline every time?
- [] Does each run record which models were used, whether the graph was consulted first, whether cacheable prefixes were preserved, and how much duplicated reading was avoided?

Baseline SKILL.md

The template below is a practical baseline for the Codex skill you described. It is designed to encode the public model/routing guidance, the graph-first workflow, and the token-efficiency prompting rules supported by the current OpenAI and Graphify docs. ²³

```
# Token Efficiency Coordinator
```

```
## Purpose
```

Use this skill whenever the task involves repo exploration, multi-file work, refactoring, debugging, code review, architecture analysis, or any request that could cause unnecessary token burn.

Primary objective:

- Minimize total token / credit spend without sacrificing correctness.
- Improve user-visible speed by routing work to smaller workers whenever possible.
- Use the graph as the default repo map so the agent does not repeatedly rediscover structure.

```
## Non-negotiable operating rules
```

1. Graph before grep.
2. Graph before broad file reads.
3. Graph before spawning multiple workers.
4. After any implemented change, update the graph or confirm the graph hook already refreshed it.

5. Use the most expensive reasoning only where ambiguity, risk, or coordination truly requires it.
6. Keep repeated instruction prefixes stable so prompt caching can work.
7. Give smaller models narrower scopes, stricter output contracts, and fewer files.

Model alias map

Treat internal shorthand as follows:

- ``frontier_deep`` = ``gpt-5.5`` with ``reasoning.effort: xhigh``
- ``frontier_balanced`` = ``gpt-5.5`` with ``reasoning.effort: medium``
- ``frontier_reviewer`` = ``gpt-5.5`` with ``reasoning.effort: high``
- ``standard_balanced`` = ``gpt-5.4`` with ``reasoning.effort: medium``
- ``light_worker`` = ``gpt-5.4-mini`` with ``reasoning.effort: low``
- ``atomic_worker`` = ``gpt-5.4-nano`` with ``reasoning.effort: none``
- ``realtime_editor`` = ``gpt-5.3-codex-spark`` for fast text-only iteration

Normalization rules:

- If someone says "5.5 extra high", interpret it as ``frontier_deep``.
- If someone says "5.4 light", interpret it as ``light_worker`` by default.
- If the task is purely mechanical classification / extraction / ranking, "5.4 light" may be downgraded to ``atomic_worker``.
- Do not default to deprecated ChatGPT-sign-in Codex models.

Hard-coded rate card

Use these values for on-the-fly cost math.

Batch / API-style price hints per 1M tokens

- ``gpt-5.5``: input 5.00, cached_input 0.50, output 30.00
- ``gpt-5.4``: input 2.50, cached_input 0.25, output 15.00
- ``gpt-5.4-mini``: input 0.75, cached_input 0.075, output 4.50
- ``gpt-5.4-nano``: input 0.20, cached_input 0.02, output 1.25

Codex credit hints per 1M tokens

- ``gpt-5.5``: input 125, cached_input 12.5, output 750
- ``gpt-5.4``: input 62.5, cached_input 6.25, output 375
- ``gpt-5.4-mini``: input 18.75, cached_input 1.875, output 113

Notes:

- Spark has separate preview limits; do not assume normal API economics for it.
- If auth mode is unclear, estimate with the Batch/API-style rate card.
- If using ChatGPT Codex credits, prefer the credit card above for comparisons.

Cost-estimation formula

Before choosing a plan, estimate:

```
`estimated_cost = input_mtok * input_rate + cached_input_mtok *  
cached_input_rate + output_mtok * output_rate`
```

When comparing options:

- Prefer the cheapest plan that is likely to meet the quality bar.
- Prefer a worker swarm only if total estimated worker cost + coordinator cost is materially below the monolithic alternative.
- Default decision threshold: choose the swarm if estimated total cost is $\leq 0.85 \times$ the monolithic ``gpt-5.5`` plan.

-
If the task is security-sensitive, architecture-critical, or highly ambiguous, quality overrides this threshold.

Mandatory graph workflow

At the start of repo-facing work:

1. Ask: "Can the graph answer this first?"
2. Use one or more of:
 - ``graphify query "<question>"``
 - ``graphify path "<symbol_a>" "<symbol_b>"``
 - ``graphify explain "<symbol_or_concept>"``
3. Extract:
 - relevant subsystems
 - connected symbols
 - likely blast radius
 - minimal file set
4. Only then decide whether file reads are needed.
5. Spawn workers using graph slices, not whole-repo context.

After edits:

- If graph hooks are installed, assume the graph may already refresh automatically.
- If not certain, run an update path such as ``graphify update`` or ``/graphify --update``.
- Never leave the repo in a state where the graph no longer represents the current implementation if future runs will depend on it.

Worker routing policy

Use ``frontier_balanced`` or ``frontier_deep`` only for:

- merging results from several workers
- resolving conflicting evidence
- architecture decisions spanning many modules
- subtle bug chains

- security or correctness audits
- final synthesis for the user
- planning when the task is underspecified or risky

Use `standard\_balanced` for:

- solid coding work that is not cheap enough for mini
- pinned GPT-5.4 workflows
- medium-complexity multi-file changes with moderate ambiguity

Use `light\_worker` for:

- repo exploration
- read-heavy scans
- blast-radius inventory
- supporting-document review
- finding likely files for a bug or feature
- summarizing individual modules for the coordinator

Use `atomic\_worker` for:

- classification
- extraction
- ranking
- de-duplication
- bucketing tests / logs / stack traces
- generating rename maps
- strict yes/no or small-schema outputs

Use `realtime\_editor` for:

- fast UI tweaks
- immediate local iteration
- focused edits where latency matters more than broad reasoning
- short "change this and stop" tasks

Prompting rules by model class

For `gpt-5.5`

Use short, outcome-first prompts.

Required structure:

- objective
- success criteria
- constraints
- evidence sources
- desired final output

Avoid:

- long legacy process scripts
- redundant instructions
- contradictory guidance

- vague "be thorough" filler

For `gpt-5.4-mini` and `gpt-5.4-nano`
Use strict task cards.

Required structure:

- exact goal
- exact scope
- allowed inputs
- allowed tools
- forbidden actions
- explicit ordered steps
- exact return schema
- stopping condition

Smaller workers must not receive:

- broad ambiguous briefs
- unnecessary repo history
- giant architecture reports
- overlapping file sets unless overlap is intentional

Standard worker task cards

Scan worker card

Goal:

Scope:

Relevant graph findings:

Files allowed to read:

Questions to answer:

Return format:

- relevant files
- key findings
- uncertainties
- recommended escalation yes/no

Atomic worker card

Goal:

Input set:

Transformation required:

Rules:

Return JSON schema:

Stopping condition:

Reviewer worker card

Goal:

Change set:

Risks to inspect:

Validation required:

Return format:

- findings
- severity
- confidence
- evidence

Escalation rules

Escalate upward when:

- a small worker encounters ambiguity it cannot resolve from the provided graph slice and files
- the blast radius grows beyond the assigned scope
- security / auth / payments / data-loss logic appears
- multiple workers disagree
- the task touches architecture, migrations, concurrency, or subtle correctness rules

Escalation ladder:

- ``atomic_worker`` -> ``light_worker``
- ``light_worker`` -> ``standard_balanced``
- ``standard_balanced`` -> ``frontier_balanced``
- ``frontier_balanced`` -> ``frontier_deep`` only when the hard case remains unresolved

De-duplication rules

- Do not let multiple workers read the same large files unless the coordinator explicitly wants independent review.
- Do not send whole reports when a graph slice plus 3-8 files is enough.
- Do not re-send stable repository rules if they already live in ``AGENTS.md``.
- Do not ask a worker to both explore broadly and implement broadly in the same task.
- Separate discovery workers from implementation workers.

Caching rules

When API-key mode is available:

- Keep the stable instruction prefix at the beginning of the request.
- Put dynamic user-specific details near the end.
- Reuse consistent worker archetypes and prompt prefixes.
- Favor repeated prompt shapes for recurring scan / extract / review tasks.

Output contract for every run

Before finishing, report:

- graph consulted first: yes/no
- models used
- reasoning levels used
- why each model was chosen
- whether workers were spawned
- estimated cost reasoning
- whether the graph was refreshed after changes
- any duplicated reads that were avoided
- any places where escalation happened

Success definition

This skill succeeds when:

- the graph is used as the primary repo map
- `gpt-5.5` is mostly coordinating, not doing every small read
- cheap workers do narrow structured work
- overall spend drops
- user-visible speed is same or better
- the quality of final answers or code changes does not regress

The most important thing to preserve as you extend this skill over time is the separation of concerns: `gpt-5.5` should behave like a routing brain and judge, not a universal worker; mini and nano workers should do cheap, well-scoped work; and Graphify should keep shrinking the discovery surface so each subsequent run starts from a map instead of from scratch. That pattern is the clearest, most defensible way to get token usage materially down without giving up user-side quality. ²⁴

¹ Subagents – Codex | OpenAI Developers

<https://developers.openai.com/codex/subagents>

² All models | OpenAI API

<https://developers.openai.com/api/docs/models/all>

³ ²⁰ ²¹ GitHub - Graphify-Labs/graphify: AI coding assistant skill (Claude Code, Codex, OpenCode, Cursor, Gemini CLI, and more). Turn any folder of code, SQL schemas, R scripts, shell scripts, docs, papers, images, or videos into a queryable knowledge graph. App code + database schema + infrastructure in one graph. · GitHub

<https://github.com/safishamsi/graphify>

⁴ ⁵ ¹⁰ ¹¹ Models – Codex | OpenAI Developers

<https://developers.openai.com/codex/models>

⁶ ¹² ¹³ ¹⁴ GPT-5.5 Model | OpenAI API

<https://developers.openai.com/api/docs/models/gpt-5.5>

⁷ ⁸ ¹⁹ ²² ²⁴ Subagents – Codex | OpenAI Developers

<https://developers.openai.com/codex/concepts/subagents>

⁹ GPT-5.4 nano Model | OpenAI API

<https://developers.openai.com/api/docs/models/gpt-5.4-nano>

15 Prompt caching | OpenAI API

<https://developers.openai.com/api/docs/guides/prompt-caching>

16 17 18 Prompt guidance | OpenAI API

<https://developers.openai.com/api/docs/guides/prompt-guidance>

23 Agent Skills – Codex | OpenAI Developers

<https://developers.openai.com/codex/skills>